

SIMATS ENGINEERING

ASSESSMENT TOOL 2

COURSE CODE: CSA5112

COURSE NAME: Cryptography and Network Security

COURSE OUTCOME COVERED

CO5: *Develop the network security strategies based on the study of viruses, worms, firewall architectures, and IDS/IPS functionalities.CO (BL5 , BL6)*

Assessment Tool Weightage: 20%

QUESTIONS: 5

TOTAL MARKS: 25

Annexure A

PSEUDOCODE WRITING TASK (ALGORITHM DESIGN)

Code of Conduct:

*I G. Srivalli (192365014) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.*

Signature of the student:

G. Srivalli

Annexure A

PSEUDOCODE WRITING TASK (ALGORITHM DESIGN)

1. Design pseudocode for a **stateful firewall system** that tracks active connections and filters packets based on session state. Evaluate how state tracking improves security compared to stateless filtering.

2. Design pseudocode for a **signature-based malware detection system** that scans files and compares them against a known signature database. Evaluate its effectiveness in detecting known threats and its limitations against new attacks.

3. Design pseudocode for a **distributed denial-of-service (DDoS) detection system** using traffic threshold analysis. Evaluate how the algorithm differentiates between legitimate high traffic and attack traffic.

4. Design pseudocode for a **network intrusion detection system using machine learning classification**. Evaluate how feature selection and training data impact detection accuracy.

5. Design pseudocode for a **secure network access control system** that authenticates users and enforces role-based access policies. Evaluate how access control mechanisms enhance overall network security.

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Problem Understanding	20%	Identifies all inputs, outputs, constraints accurately	Minor missing elements	Partial understanding	Incorrect interpretation
Algorithm Design	30%	Complete, correct, and logically sound solution	Minor logical gaps	Partially correct logic	Incorrect approach
Use of Control Structures	20%	Efficient and appropriate use of loops, conditions, functions/modules	Minor inefficiencies	Limited or basic usage	Incorrect or missing constructs
Pseudocode Structure	10%	Well-structured, clear, consistent format, easy to follow	Mostly clear with minor issues	Difficult to follow in parts	Poorly structured and unclear
Completeness	20%	Handles all cases; optimized approach	Handles most cases; reasonable efficiency	Handles basic cases only	Incomplete or inefficient

Question 1: Stateful Firewall System Design & Evaluation

1. System Problem Understanding & Architecture:

A stateful firewall monitors the state of active network connections (such as TCP handshakes, established sessions, and UDP virtual state tracking) and uses this contextual state information to decide whether incoming and outgoing packets should be allowed or blocked. Unlike stateless firewalls that inspect packets in isolation based purely on static headers, a stateful firewall maintains a dynamically updated Connection State Table (State Table).

System Parameters:

Input Packet: Source IP, Source Port, Destination IP, Destination Port, Protocol, TCP Flags (SYN, ACK, FIN, RST), Payload.

Output Action: ALLOW, DROP, or REJECT, along with appropriate log entries and dynamic state table updates.

Constraints: Maintain session state (LISTEN, SYN_SENT, SYN_RCVD, ESTABLISHED, FIN_WAIT, CLOSED) and process packets in line without inducing significant latency.

2. Pseudocode Implementation:

```
// ALGORITHM: Stateful Firewall Packet Processing Engine
STRUCT ConnectionState:
    src_ip: String, src_port: Integer
    dst_ip: String, dst_port: Integer
    protocol: String, state: String
    last_timestamp: DateTime, seq_num: Integer, ack_num: Integer

GLOBAL StateTable = DICTIONARY() // Key: (src_ip, src_port, dst_ip,
dst_port, protocol)
GLOBAL RuleSet = LOAD_SECURITY_RULES()

FUNCTION ProcessPacket(packet):
    conn_key = (packet.src_ip, packet.src_port, packet.dst_ip,
packet.dst_port, packet.protocol)
    rev_conn_key = (packet.dst_ip, packet.dst_port, packet.src_ip,
packet.src_port, packet.protocol)

    IF packet.protocol == 'TCP':
        RETURN ProcessTCPPacket(packet, conn_key, rev_conn_key)
    ELSE IF packet.protocol == 'UDP':
        RETURN ProcessUDPPacket(packet, conn_key, rev_conn_key)
    ELSE:
        RETURN EvaluateStatelessRules(packet)
```

```

FUNCTION ProcessTCPPacket(packet, conn_key, rev_conn_key):
    // Case 1: Outgoing new connection request (SYN packet)
    IF packet.flags == 'SYN' AND NOT StateTable.CONTAINS(conn_key)
    AND NOT StateTable.CONTAINS(rev_conn_key):
        IF EvaluateSecurityPolicy(packet) == ALLOW:
            StateTable[conn_key] = ConnectionState(packet.src_ip,
            packet.src_port, packet.dst_ip, packet.dst_port, 'TCP', 'SYN_SENT',
            NOW(), packet.seq, 0)
            LOG_ACTION('ALLOW_NEW_SYN', packet)
            RETURN ACTION_ALLOW
        ELSE:
            LOG_ACTION('DROP_POLICY_VIOLATION', packet)
            RETURN ACTION_DROP

    // Case 2: Inbound SYN-ACK response matching existing SYN_SENT
    state
    ELSE IF packet.flags == ('SYN' + 'ACK') AND
    StateTable.CONTAINS(rev_conn_key):
        session = StateTable[rev_conn_key]
        IF session.state == 'SYN_SENT' AND packet.ack ==
        session.seq_num + 1:
            session.state = 'SYN_RCVD'
            session.last_timestamp = NOW()
            LOG_ACTION('ALLOW_SYN_ACK', packet)
            RETURN ACTION_ALLOW

    // Case 3: Final ACK completing 3-way handshake
    ELSE IF packet.flags == 'ACK' AND
    StateTable.CONTAINS(conn_key):
        session = StateTable[conn_key]
        IF session.state == 'SYN_RCVD' OR session.state ==
        'SYN_SENT':
            session.state = 'ESTABLISHED'
            session.last_timestamp = NOW()
            LOG_ACTION('ESTABLISHED_SESSION', packet)
            RETURN ACTION_ALLOW

    // Case 4: Existing Established Connection Data Traffic
    ELSE IF StateTable.CONTAINS(conn_key) OR
    StateTable.CONTAINS(rev_conn_key):
        active_key = conn_key IF StateTable.CONTAINS(conn_key) ELSE
        rev_conn_key
        session = StateTable[active_key]
        IF session.state == 'ESTABLISHED':
            IF packet.flags CONTAINS 'FIN':

```

```
        session.state = 'FIN_WAIT'
    ELSE IF packet.flags CONTAINS 'RST':
        StateTable.REMOVE(active_key)
        session.last_timestamp = NOW()
    RETURN ACTION_ALLOW

// Default Fallback: Unsolicited / Invalid Packet
LOG_ACTION('DROP_UNSOLICITED_STATE', packet)
RETURN ACTION_DROP
```

3. Critical Evaluation & Security Comparison:

Stateful firewalls offer substantial security enhancements over traditional stateless (packet-filtering) firewalls. The table below outlines the core differences:

Security Feature	Stateless Filtering	Stateful Inspection
Contextual Awareness	Evaluates each packet independently based purely on header rules.	Tracks entire session lifecycle across multiple TCP/UDP packets.
Handshake Validation	Cannot verify if a packet belongs to an established handshake.	Enforces full 3-way handshake (SYN -> SYN-ACK -> ACK) before allowing data.
Port Opening / Rules	Requires broad, static open ports (e.g., high ephemeral ports open).	Dynamically opens ephemeral ports only for validated active sessions.
Attack Mitigation	Vulnerable to IP spoofing, ACK floods, and out-of-order scans.	Defeats ACK scan, TCP sequence attacks, and unsolicited inbound probes.

Question 2: Signature-Based Malware Detection System

Design pseudocode for a signature-based malware detection system that scans files and compares them against a known signature database. Evaluate its effectiveness in detecting known threats and its limitations against new attacks.

1. Problem Understanding & Architecture:

A signature-based malware detection system computes cryptographic hashes (e.g., SHA-256) and extracts static byte sequences (pattern matchers) from target executable files. It then queries a centralized signature repository. If a match is detected, the file is identified as malicious and isolated.

System Parameters:

Input: Target file object, byte streams, and Known Malware Database (Hash table + YARA rule byte patterns).

Output: Threat Classification (CLEAN, MALICIOUS, SUSPICIOUS), Threat Metadata, and Isolation Action.

Constraints: High-throughput file scanning, low false-positive rates, efficient memory lookup.

2. Pseudocode Implementation:

```
// ALGORITHM: Signature-Based Malware Detection & Scanning Engine
STRUCT MalwareSignature:
    sig_id: String, malware_name: String
    sha256_hash: String, byte_sequence: ByteStream, risk_level:
String

GLOBAL HashDatabase = LOAD_KNOWN_HASHES()      // Hash map lookup
O(1)
GLOBAL PatternDatabase = LOAD_YARA_PATTERNS() // Aho-Corasick
Automaton

FUNCTION ScanFile(file_path):
    IF NOT FILE_EXISTS(file_path):
        RETURN ERROR('File Not Found')

    file_bytes = READ_FILE_BYTES(file_path)
    file_hash = COMPUTE_SHA256(file_bytes)

    // Phase 1: Cryptographic Hash Lookup (Exact Match)
    IF HashDatabase.CONTAINS_KEY(file_hash):
        threat_info = HashDatabase[file_hash]
```

```

        QUARANTINE_FILE(file_path)
        LOG_DETECTION('EXACT_HASH_MATCH', threat_info.malware_name,
file_path)
        RETURN RESULT_MALICIOUS(threat_info)

    // Phase 2: Static Byte Pattern Sequence Matching
    matched_patterns = AHO_CORASICK_SEARCH(file_bytes,
PatternDatabase)
    IF LENGTH(matched_patterns) > 0:
        FOR EACH pattern IN matched_patterns:
            IF pattern.confidence >= 0.85:
                QUARANTINE_FILE(file_path)
                LOG_DETECTION('PATTERN_MATCH', pattern.rule_name,
file_path)
                RETURN RESULT_MALICIOUS(pattern)

    // Phase 3: Fuzzy Hash / SSDEEP Matching (Near-Match for
Variants)
    fuzzy_hash = COMPUTE_SSDEEP(file_bytes)
    similarity = COMPARE_SSDEEP_DB(fuzzy_hash, HashDatabase)
    IF similarity > 80%:
        FLAG_FOR_HEURISTIC_ANALYSIS(file_path)
        RETURN RESULT_SUSPICIOUS('Fuzzy match similarity: ' +
similarity)

    LOG_INFO('FILE_CLEAN', file_path)
    RETURN RESULT_CLEAN

FUNCTION QUARANTINE_FILE(file_path):
    ENCRYPT_FILE(file_path, KEY_QUARANTINE)
    MOVE_TO_DIR(file_path, '/var/quarantine/')

```

3. Critical Evaluation & Efficacy Analysis:

Signature-based detection remains a cornerstone of endpoint detection and response (EDR) due to its precision and minimal computational overhead for known threats.

Strengths & Effectiveness against Known Threats:

- 1. Deterministic Accuracy:** Produces near-zero false positive rates when matching exact SHA-256 hashes.
- 2. High Efficiency & Speed:** Hash table lookups operate in $O(1)$ time complexity, permitting real-time file scanning on access.

3. Low Resource Overhead: Straightforward verification without complex machine learning model training or behavioral sandboxing.

Vulnerabilities & Limitations against Advanced Attacks:

1. Zero-Day Blindspot: Completely ineffective against Zero-Day malware, as signature repositories lack patterns for novel threats.

2. Evasion via Polymorphism: Attackers easily alter file hashes using polymorphic and metamorphic engines (e.g., adding arbitrary junk bytes, reordering instructions, or altering encryption wrappers) without changing malicious functionality.

3. Database Scalability Strain: The signature database expands exponentially, requiring massive memory footprints and constant signature update pushes to endpoints.

Question 3: Distributed Denial-of-Service (DDoS) Detection System

Design pseudocode for a distributed denial-of-service (DDoS) detection system using traffic threshold analysis. Evaluate how the algorithm differentiates between legitimate high traffic and attack traffic.

1. Problem Understanding & Architecture

A DDoS detection engine analyzes continuous network telemetry (e.g., packets per second [PPS], SYN ratio, entropy of source IPs) across rolling time windows. It uses adaptive thresholding and behavioral indicators to distinguish malicious botnet volumetric floods from legitimate traffic spikes (e.g., flash crowds during breaking news or sales events).

System Parameters:

Input Data Stream: Streaming network packet data, ingress interfaces, historical traffic baselines.

Output Actions: Alert Level (NORMAL, WARNING, MITIGATE_DDOS), Rate-Limiting rules, BGP FlowSpec diversion commands.

Constraints: Low latency window aggregation (e.g., 10-second sliding windows) with low memory footprint.

2. Pseudocode Implementation:

```
// ALGORITHM: Sliding-Window DDoS Detection & Traffic Threshold Analyzer
STRUCT TrafficMetrics:
    total_pps: Integer, total_bps: Integer
    syn_count: Integer, ack_count: Integer
    unique_src_ips: SET, ip_entropy: Float

GLOBAL WindowSize = 10 // seconds
GLOBAL Baseline_PPS = 50000, Baseline_Entropy = 4.5

FUNCTION MonitorTrafficStream(packet_stream):
    metrics = InitializeMetrics()
    start_time = NOW()

    FOR EACH packet IN packet_stream:
        metrics.total_pps += 1
        metrics.total_bps += packet.length
        metrics.unique_src_ips.ADD(packet.src_ip)

        IF packet.tcp_flags CONTAINS 'SYN':
```

```

        metrics.syn_count += 1
    ELSE IF packet.tcp_flags CONTAINS 'ACK':
        metrics.ack_count += 1

    IF NOW() - start_time >= WindowSize:
        metrics.ip_entropy =
CALCULATE_ENTROPY(metrics.unique_src_ips)
        EVALUATE_DDOS_THREAT(metrics)
        metrics = InitializeMetrics()
        start_time = NOW()

FUNCTION EVALUATE_DDOS_THREAT(m):
    pps_ratio = m.total_pps / Baseline_PPS
    syn_ack_ratio = (m.syn_count + 1) / (m.ack_count + 1)

    // Condition 1: Volumetric Surge + Low Source Entropy (Botnet /
    Spoofed Flood)
    IF pps_ratio > 4.0 AND m.ip_entropy < (Baseline_Entropy * 0.4):
        TRIGGER_ALERT('CRITICAL_DDOS_FLOOD_DETECTED', 'Volumetric
flood with concentrated low-entropy IPs')
        ACTIVATE_BGP_SCRUBBING()
        ENFORCE_RATE_LIMITING(m.unique_src_ips)

    // Condition 2: SYN Flood Attack Detection
    ELSE IF syn_ack_ratio > 10.0 AND m.syn_count > 10000:
        TRIGGER_ALERT('CRITICAL_SYN_FLOOD', 'High SYN-to-ACK ratio
detected')
        ENABLE_SYN_COOKIES()

    // Condition 3: Legitimate Flash Crowd (High PPS, High Entropy,
    Normal Handshakes)
    ELSE IF pps_ratio > 3.0 AND m.ip_entropy >= (Baseline_Entropy *
0.9) AND syn_ack_ratio < 2.0:
        LOG_EVENT('FLASH_CROWD_DETECTED', 'Legitimate high volume
traffic. Auto-scaling server resources.')
        SCALE_INFRASTRUCTURE_RESOURCES()

    ELSE:
        UPDATE_DYNAMIC_BASELINE(m)

```

3. Differentiation Analysis: Flash Crowds vs. Attack Traffic:

A critical challenge in network security is avoiding false-positive mitigations during legitimate peak traffic events. The table below illustrates how behavioral indicators distinguish these scenarios:

Traffic Metric	Legitimate Flash Crowd	DDoS Attack Traffic
IP Distribution & Entropy	High entropy; diverse, well-distributed real user IPs worldwide.	Low entropy; clustered IPs, subnet spoofing, or repetitive botnet agents.
Protocol Behavior	Complete TCP handshakes, valid HTTP GET/POST headers, expected cookie responses.	Incomplete handshakes (SYN floods), malformed requests, asymmetric payload ratios.
Request Patterns	Varied session durations, sequential page navigation, standard user think time.	Identical request rates, repeated URL hammering, static User-Agent strings.
Response Handling	Clients accept cookies, process JavaScript redirects, and complete CAPTCHAs.	Automated scripts fail JS challenges, drop cookies, or ignore HTTP redirects.

Question 4: Machine Learning-Based Network Intrusion Detection System (NIDS)

Design pseudocode for a network intrusion detection system using machine learning classification. Evaluate how feature selection and training data impact detection accuracy.

1. Problem Understanding & Architecture:

A Machine Learning NIDS processes raw network packet captures (e.g., PCAP/NetFlow), transforms them into feature vectors (e.g., packet lengths, flow durations, flag counts), normalizes the data, and classifies traffic as BENIGN or MALICIOUS using a trained model (e.g., Random Forest or XGBoost).

System Parameters:

Input: Raw packet stream, feature definitions, trained classification model weights.

Output: Prediction Class (0: Normal, 1: Probe, 2: DoS, 3: R2L/U2R), Confidence Score, Security Alert.

Constraints: Real-time feature extraction under high network throughput with minimal classification delay.

2. Pseudocode Implementation:

```
// ALGORITHM: ML-Based Network Intrusion Detection Pipeline
IMPORT Model, Scaler, FeatureSelector

FUNCTION TrainNIDSModel(training_dataset):
    // Step 1: Preprocessing & Cleaning
    clean_data = REMOVE_NULLS_AND_DUPLICATES(training_dataset)
    encoded_data = ENCODE_CATEGORICAL_FEATURES(clean_data,
['protocol', 'service', 'flag'])
    // Step 2: Feature Selection (Mutual Information / Random
Forest Importance)
    X_raw = encoded_data.DROP('label')
    y = encoded_data['label']
    selected_features = SELECT_K_BEST_FEATURES(X_raw, y, k=15)
    X_selected = X_raw[selected_features]
    // Step 3: Feature Scaling
    Scaler = STANDARD_SCALER().FIT(X_selected)
    X_scaled = Scaler.TRANSFORM(X_selected)
    // Step 4: Model Training with Cross-Validation
    Model = TRAIN_RANDOM_FOREST(X_scaled, y, trees=100,
max_depth=15)
    EVALUATE_MODEL_METRICS(Model, X_scaled, y)
```

```

    RETURN Model, Scaler, selected_features
FUNCTION PredictNetworkFlow(raw_flow_bytes):
    // Extract key metrics from flow stream
    flow_features = EXTRACT_FLOW_FEATURES(raw_flow_bytes)
    // Metrics: [duration, src_bytes, dst_bytes, wrong_fragment,
count, srv_count, etc.]
    // Apply learned selection and scaling
    filtered_features = FILTER_FEATURES(flow_features,
selected_features)
    scaled_features = Scaler.TRANSFORM(filtered_features)
    prediction = Model.PREDICT(scaled_features)
    probability = Model.PREDICT_PROBA(scaled_features)
    IF prediction != 'BENIGN' AND probability > 0.80:
        GENERATE_INTRUSION_ALERT(prediction, probability,
flow_features)
        BLOCK_SOURCE_IP(flow_features.src_ip)
        RETURN ACTION_BLOCK
    ELSE:
        RETURN ACTION_ALLOW

```

3. Evaluation of Feature Selection & Training Data Impact:

Impact of Feature Selection:

Dimensionality Reduction: High-dimensional feature spaces contain redundant and noisy features (e.g., source port numbers). Selecting high-gain features via Mutual Information or Recursive Feature Elimination (RFE) reduces model training/inference latency and prevents over-fitting.

Improved Generalization: By isolating core behavioral features (e.g., flow byte rate, TCP flag distributions, error packet ratios), the model learns fundamental attack patterns rather than memorizing noise.

Impact of Training Data Quality:

Class Imbalance Bias: If training sets consist predominantly of benign traffic (e.g., 99% normal, 1% attack), models become biased toward the majority class, yielding high overall accuracy but missing critical attacks. Techniques like SMOTE (Synthetic Minority Over-sampling Technique) or class weighting must be applied.

Dataset Staleness & Drift: Models trained on obsolete datasets (e.g., KDD Cup 99) fail to recognize modern attack vectors like cloud container escapes, IoT botnets, or encrypted TLS tunneling attacks.

Question 5: Secure Network Access Control (NAC) & RBAC System

Design pseudocode for a secure network access control system that authenticates users and enforces role-based access policies. Evaluate how access control mechanisms enhance overall network security.

1. Problem Understanding & Architecture:

A Network Access Control (NAC) system validates user identities, multi-factor authentication (MFA) tokens, and device posture compliance before granting dynamic network access. Upon authentication, Role-Based Access Control (RBAC) maps the user to a defined role (e.g., Admin, Employee, Guest) and applies fine-grained Access Control Lists (ACLs) and VLAN assignments.

System Parameters:

Input: User Credentials, MFA Token, Device MAC address, Security Posture status, Requested Resource.

Output: Access Status (GRANTED/DENIED), Assigned VLAN ID, Dynamic Firewall ACLs.

Constraints: Zero-Trust default-deny posture; strict lease expiration and session timeouts.

2. Pseudocode Implementation:

```
// ALGORITHM: Secure Network Access Control (NAC) & RBAC Policy Engine
STRUCT UserSession:
    username: String, role: String
    assigned_vlan: Integer, granted_permissions: LIST
    session_token: String, expiry_time: DateTime

GLOBAL UserDirectory = LOAD_LDAP_ACTIVE_DIRECTORY()
GLOBAL PolicyTable = LOAD_RBAC_POLICY_MATRIX()

FUNCTION AuthenticateAndAuthorizeUser(username, password, mfa_code,
device_info):
    // Step 1: Identity Verification
    user = UserDirectory.FIND(username)
    IF user == NULL OR NOT VERIFY_HASH(password,
user.password_hash):
        LOG_SECURITY_AUDIT('AUTH_FAILED_INVALID_CREDS', username)
        RETURN ACCESS_DENIED('Invalid Credentials')

    // Step 2: Multi-Factor Authentication (MFA) Check
    IF NOT VERIFY_MFA_TOKEN(user.mfa_secret, mfa_code):
        LOG_SECURITY_AUDIT('AUTH_FAILED_MFA', username)
        RETURN ACCESS_DENIED('MFA Verification Failed')
```

```

// Step 3: Device Posture Assessment (Compliance Check)
IF NOT CheckDeviceCompliance(device_info):
    ASSIGN_VLAN(device_info.mac, VLAN_QUARANTINE)
    LOG_SECURITY_AUDIT('NON_COMPLIANT_DEVICE', username)
    RETURN ACCESS_RESTRICTED('Device non-compliant. Redirected
to Quarantine.')

// Step 4: Role-Based Access Policy Assignment
user_role = user.role
role_policy = PolicyTable[user_role]

session = UserSession(
    username = username,
    role = user_role,
    assigned_vlan = role_policy.vlan_id,
    granted_permissions = role_policy.permissions,
    session_token = GENERATE_SECURE_TOKEN(),
    expiry_time = NOW() + HOURS(8)
)
ENFORCE_DYNAMIC_VLAN(device_info.mac, session.assigned_vlan)
APPLY_FIREWALL_ACL(device_info.ip, session.granted_permissions)
LOG_SECURITY_AUDIT('AUTH_SUCCESS', username)
RETURN ACCESS_GRANTED(session)

FUNCTION EnforceRBAC(session, target_resource, requested_action):
    IF NOW() > session.expiry_time:
        REVOKE_ACCESS(session)
        RETURN ACCESS_DENIED('Session Expired')

    permission_key = target_resource + ':' + requested_action
    IF session.granted_permissions.CONTAINS(permission_key):
        RETURN PERMIT_ACTION
    ELSE:
        LOG_SECURITY_AUDIT('UNAUTHORIZED_ACCESS_ATTEMPT',
session.username)
        RETURN ACCESS_DENIED('Insufficient Privileges')

```

3. Evaluation of Access Control Security Enhancements:

1. Containment & Blast Radius Reduction:

By enforcing strict VLAN segregation and dynamic network segmentation, compromised guest or employee devices cannot move laterally across internal subnets to target critical database servers.

2. Implementation of Principle of Least Privilege (PoLP):

Users receive only the absolute minimum permissions required for their explicit job function. Temporary role elevations require explicit re-authentication and administrative approval.

3. Continuous Posture & Identity Enforcement:

NAC ensures unpatched, non-compliant devices (e.g., missing antivirus, disabled firewall) are quarantined prior to gaining network access, protecting the infrastructure from insider threat vectors.